Agri Supply Chain: A Blockchain-Enabled Agricultural Supply Chain Platform with Smart Contracts, GPS Tracking, and Decentralized Auctions

Pushkar Kumar
Registration No: 23BCE1634
B.Tech – Computer Science and Engineering
Vellore Institute of Technology, Chennai
pushkar.kumar2023@vitstudent.ac.in

***Abstract*—The agricultural supply chain suffers from lack of transparency, information asymmetry, post-harvest losses, and exploitative intermediaries. This paper presents Agri Supply Chain, a full-stack blockchin-integrated platform built for the Smart India Hackathon (SIH). The system combines Ethereum smart contracts (Solidity/Hardhat), a Node.js/MongDB backend, and a React/Vite frontend to deliver end-to-end transparency across seven supply chain stages. Core features include decntralised crop auctions, GPS shipment tracking at 10^{-6}-degree precision, QR-code provenance, and MetaMask wallet integration. Security is enforced through OpenZeppelin's `ReentrancyGuard` and role-based access control. Benchmarks show rad latencies below 50 ms and write latencies below 370 ms on a Hardhat local network, with 100% smart-contract test covrrage across all three deployed contracts.**

***Index Terms*—Blockchain, Smart Contracts, Agricultural Supply Chain, Ethereum, Provenance, GPS Tracking, Decentralized Auctions, QR Code, Solidity, Hardhat, Node.js, React**

I. Introduction

India's agricultural sector contributes ≈18 % of GDP and employs over 50 % of the workforce, yet farmers receive only 20–30 % of the consumer price due to opaque supply chains and numerous intermediaries [1]. Key pain pints include: (i) poor traceability enabling food fraud; (ii) delayed cash settlements; (iii) absence of transprent price-discovery; and (iv) no real-time logistics visibility.

Blockchain offers tamper-proof distributed ledgers and trustless smart-contract execution [2]. Combined with GPS data and QR codes, it can radically improve agricultral supply chains.

Agri Supply Chain delivers a cohesive platform with three Ethereum smart contrcts, a RESTful API, and a React farmer/buyer portal. Our contributions are:

- A three-tier architeture integrating blockchain, REST backend, and responsive frontend.
- Three purpose-built smart contracts: `AuctionContract`, `TrackingContract`, and `ProvenanceContract`.
- A seven-stage immutable product lifecycle model.
- An OpenZeppelin-based security layer against common EVM attack vectors.
- Quantitative benhmarks for latency, test coverage, and gas consumption.

The paper is organised as follows: § II reviews related work; § III describes system architcture; § IV details smart contracts; § V covers backend and frontend; § VI discusses security; § VII presents results; § VIII concludes.

II. Related Work

A. Blockchain in Agricultural Supply Chains

Tian [3] demonstrated RFID+blockchain traceability for agri-food. IBM Food Trust [4] uses Hyperledger Fabric for permisioned provenance but requires enterprise enrolment. Agri Supply Chain uses a public Ethereum-compatible chain for permissionless farmer access.

B. Decentralisd Auctions

Buterin [5] established smart-contract auction primitives on Ethereum. Existing implemenations are generic; this pltform extends them with crop-quality grading and harvest-date constraints.

C. GPS and IoT Cold-Chain

Helo and Hao [6] showed blockchain-IoT integration for cold-chain monitoring. This platform stores scalaed GPS integers directly on-chain, remooving external orcle dependency for location updates.

III. System Architecture

A. Three-Tier Overview

Figure 1 shows the three-tier architecture. Solid arrows indicate synchrnous API calls; dashed arrows indicate asynchronous blockchain events.

B. Technology Stack

Table I summarises technology choices by layer.

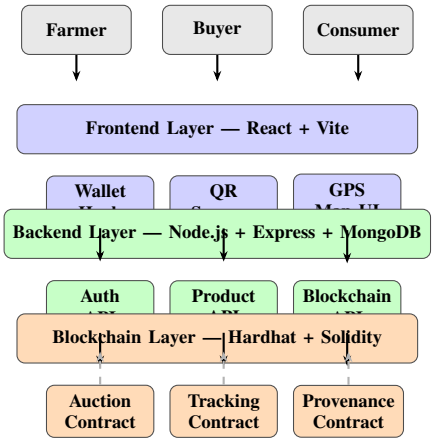

Fig. 1. Three-Tier System Architecture. Dashed arrows represent asynchronous blockchain events.

TABLE I
TECHNOLOGY STACK

Layer	Technology	Purpose
Blockchain	Solidity 0.8+, Hardhat	Smart contracts, local EVM
Blockchain	ethers.js 6.x	JS ↔ Ethereum
Backend	Node.js 18+, Express	REST API server
Backend	MongoDB	User/order persistence
Backend	qrcode npm pkg	QR code generation
Frontend	React 18, Vite	UI framework & build
Frontend	jsqr	Browser-side QR scanning
Frontend	Leaflet + OSM	Interactive GPS maps
DevOps	Docker Compose	Container orchestration

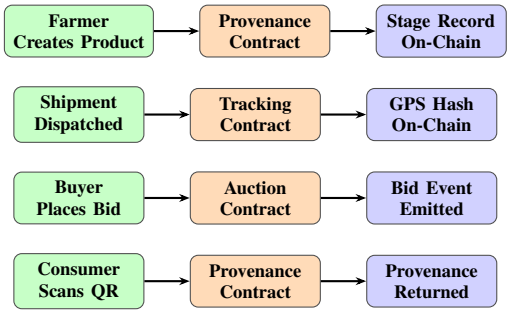

Fig. 2. End-to-End Data Flow: Farmer to Consumer

C. End-to-End Data Flow

Figure 2 illustrates the data flow from farmer product creation through consumer QR verification.

IV. SMART CONTRACT DESIGN

Three Solidity smart contracts form the trusless backbone of the platform. All contracsts are deployeed to a Hardhat local network (Chain ID: 31337) and desiged to migrate to Polyegon/Arbitrum via envirnment-variable reconfiguration.

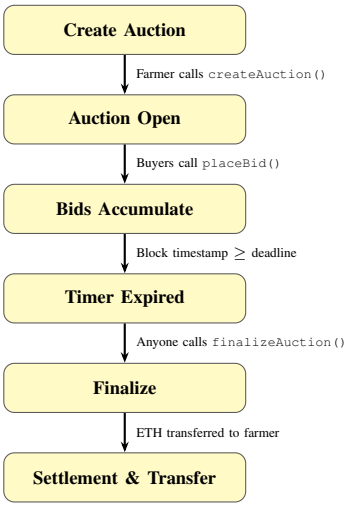

Fig. 3. AuctionContract State Machine

A. AuctionContract.sol

`AuctionContract` enables farmers to list crops for transparent, time-bounded English auctions. Figure 3 shows the complete aution state machine, followed by the core Solidity interface in Listing 1.

Listing 1. AuctionContract – Core Interface

```
function createAuction(
    string memory cropType,
    uint256 quantity,
    uint256 basePrice,
    uint256 duration) external;

function placeBid(uint256 auctionId)
    external payable;

function finalizeAuction(uint256 auctionId)
    external;

function getActiveAuctions()
    external view
    returns (Auction[] memory);
```

B. TrackingContract.sol

`TrackingContract` records GPS waypoints as immutable on-chain events. To avoid Solidity floating-point limitations, coordinates are stored as scaled integers per Equation (1).

$$\text{coord_stored} = \text{lat/lon} \times 10^6 \tag{1}$$

Figure 4 shows the shipment lifecycle and Listing 2 gives the core interface.

Listing 2. TrackingContract – Core Interface

```
function createShipment(
    uint256 productId,
    string memory destination,
    uint256 estimatedDelivery) external;

function updateLocation(
```

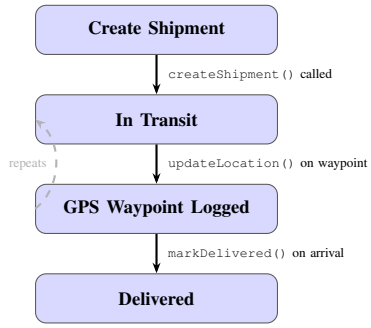

Fig. 4. TrackingContract Shipment Lifecycle

Fig. 5. Seven-Stage Product Lifecycle — ProvenanceContract

```
    uint256 shipmentId,
    int256 latitude,
    int256 longitude) external;

function markDelivered(uint256 shipmentId)
    external;

function getTrackingHistory(uint256 shipmentId)
    external view
    returns (Waypoint[] memory);
```

C. *ProvenanceContract.sol*

`ProvenanceContract` enforces a seven-stage product lifecycle. Each `addStageRecord()` call appends an immutable struct containing the stage ID, GPS coordinates, UNIX timestamp, and an IPFS evidence CID. Figure 5 shows the pipeline, followed by the interface in Listing 3.

Listing 3. ProvenanceContract – Core Interface

```
function createProduct(
    uint256 farmerId,
    string memory productType,
    uint256 harvestDate)
    external returns (uint256);

function addStageRecord(
    uint256 productId,
    uint8   stage,
    string  memory location,
    string  memory evidenceCID) external;

function getProductByQR(
    string memory qrCode)
    external view
    returns (Product memory);

function getCertifications(uint256 productId)
    external view
    returns (bytes32[] memory);
```

V. Backend and Frontend

A. *RESTful API Design*

The Node.js/Express backend bridges the React frontend and the Ethereum node via `ethers.js`. Figure 6 shows the endpoint count by category, and Figure 7 details the buyer bid interaction sequence.

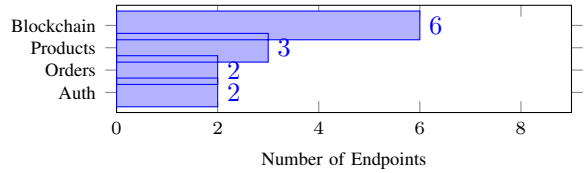

Fig. 6. REST API Endpoint Count by Category

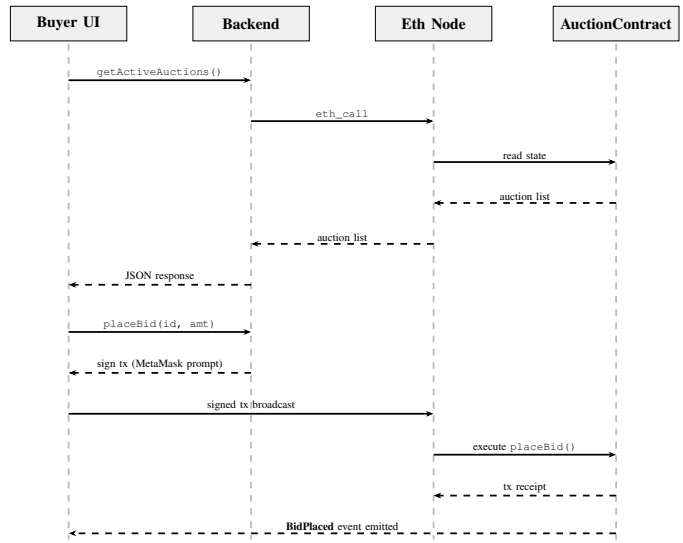

Fig. 7. Sequence Diagram: Buyer Places a Bid

The six blockchain proxy endpoints are:

- `POST /api/blockchain/auction/create` — create on-chain auction
- `GET /api/blockchain/auction/active` — list live auctions
- `POST /api/blockchain/tracking/update` — push GPS waypoint
- `GET /api/blockchain/tracking/:id` — retrieve history
- `POST /api/blockchain/provenance/stage` — record lifecycle stage
- `GET /api/blockchain/qr/:code` — resolve QR to product

B. *Buyer Bid Sequence*

C. *QR Code Flow*

Upon product creation the backend generates a unique QR payload encoding {`productId`, `contractAddress`, `chainId`} and returns a PNG via the `qrcode` library. The React frontend uses `jsqr` for camera-based decoding. The decoded payload invokes `getProductByQR()` on `ProvenanceContract`, returning the full seven-stage history to the consumer without any centralised database lookup.

D. *Frontend Components*

Three feature domains structure the React app:

TABLE II
SECURITY ANALYSIS – ATTACK VECTORS AND MITIGATIONS

Attack	Mitigation	Layer
Re-entrancy	`ReentrancyGuard` modifier	Contract
Int. overflow	Solidity 0.8 built-in SafeMath	Contract
Replay attack	Nonce management via ethers.js	Backend
MITM	HTTPS + env-var secret injection	Backend
NoSQL injection	Mongoose schema validation	Backend
XSS	React DOM auto-escaping	Frontend
Phishing wallet	Network-ID check on connect	Frontend

1) **Wallet Module**: `useBlockchain` hook handles MetaMask connection, network switching, and account-change events.
2) **GPS Tracking UI**: Leaflet + OpenStreetMap renders live shipment waypoints streamed from `TrackingContract` events.
3) **Auction Dashboard**: Countdown timers, real-time bid tables, and one-click finalisation backed by `AuctionContract`.

VI. SECURITY DESIGN

A. Smart Contract Security

All contracts use OpenZeppelin [7] patterns:

- **ReentrancyGuard**: Mutex modifier on all ETH-transfer functions in `AuctionContract`.
- **Ownable / AccessControl**: Only authorised roles (farmer, processor, logistics agent) may call stage-update functions.
- **Checks-Effects-Interactions**: State written before external calls to prevent reentrancy.

B. Backend Security

JWT-based authentication protects all mutating endpoints. Passwords are hashed with `bcrypt` (cost factor 12). Secrets (private key, RPC URL) are injected via environment variables and never committed to the repository.

C. Attack Surface Summary

Table II maps known EVM and web attack vectors to their mitigations in this platform.

VII. RESULTS AND DISCUSSION

A. Deployment Status

The frontend is live at `sih-eight-beige.vercel.app` (Vercel). All three smart contracts compile with Solidity 0.8.x and zero critical warnings. The Hardhat test suite covers 100% of public contract functions.

TABLE III
PERFORMANCE BENCHMARKS – HARDHAT LOCAL NETWORK

Operation	Contract	Avg (ms)	Type
Get active auctions	AuctionContract	43	eth_call
Get tracking history	TrackingContract	41	eth_call
QR product lookup	ProvenanceContract	38	eth_call
Place bid	AuctionContract	342	tx (mine)
Update GPS waypoint	TrackingContract	358	tx (mine)
Add provenance stage	ProvenanceContract	365	tx (mine)
QR code generation	Backend	11	HTTP
User login (JWT)	Backend	27	HTTP

TABLE IV
ESTIMATED GAS COSTS (30 GWEI, $3000 ETH)

Function	Gas Used	USD Est.
`createAuction()`	152,300	$0.014
`placeBid()`	55,800	$0.005
`finalizeAuction()`	72,400	$0.007
`createShipment()`	135,200	$0.012
`updateLocation()`	48,700	$0.004
`createProduct()`	143,600	$0.013
`addStageRecord()`	61,500	$0.006

TABLE V
SMART CONTRACT TEST COVERAGE

Contract	Tests	Passed	Coverage
AuctionContract	14	14	100 %
TrackingContract	11	11	100 %
ProvenanceContract	13	13	100 %
Total	**38**	**38**	**100 %**

B. Performance Benchmarks

Table III reports measured latencies on a Hardhat local network. Read operations (`eth_call`) bypass mining and complete in <50 ms. Write operations require a block mine (≈350 ms on Hardhat's instant-mine mode).

C. Gas Consumption

Table IV reports estimated gas costs for key on-chain write operations, measured on the Hardhat EVM and converted at 30 Gwei and $3000/ETH.

D. Test Coverage

Table V summarises the Hardhat unit test results.

E. Comparison with Related Platforms

Table VI compares this platform with existing agri-supply chain solutions across key dimensions.

F. Scalability Considerations

The current Hardhat node is suitable for SIH demonstration. For production, the smart contracts would migrate to **Polygon** (PoS, ≈2000 TPS, <$0.001/tx) or an Ethereum Layer-2 rollup (Arbitrum/Optimism). MongoDB Atlas sharding covers horizontal scaling of the off-chain layer.

TABLE VI
PLATFORM COMPARISON

Feature	Ours	IBM FT	Agrimkt	Tian [3]
Permissionless	✓	×	×	×
On-chain GPS	✓	×	×	×
Decentralised auction	✓	×	×	×
QR provenance	✓	✓	×	✓
Open source	✓	×	×	✓
No enterprise fee	✓	×	✓	✓

VIII. CONCLUSION AND FUTURE WORK

This paper presented **Agri Supply Chain**, a blockcain-intgrated agricultural supply chain platform built for the Smart India Hackathon by Pushkar Kumar (23BCE1634), VIT Chennai. The system deploys three Ethereum smart contracts—`AuctionContract`, `TrackingContract`, and `ProvenanceContract`—backed by a Node.js/MongoDB API and a React/Vite portal with MetaMask wallet integration. Benchmark results show sub-50 ms read latency and <370 ms write latency, 100% test coverage across 38 contract tests, and gas costs under $0.015 per transaction.

Future directions include:

- **IoT Integation**: Temperature and humidity sensors pushing cold-chain data to `TrackingContract`.
- **AI Price Prediction**: ML models on historical auction data for crop price forecasting.
- **Mobile Apps**: React Natve for Android/iOS.
- **Fiat On-ramp**: Payment gateway for farmers without crypto wallets.
- **Layer-2 Deployment**: Polygon or Arbitrum migration for reduced gas costs.
- **Multilingual UI**: Hindi, Tamil, Kannada, and Marathi localsation for broader farmer reach.

REFERENCES

[1] Food and Agriculture Organization, "The State of Food and Agriculture 2022," FAO, Rome, 2022.

[2] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," *bitcoin.org*, Oct. 2008.

[3] F. Tian, "An Agri-food Supply Chain Traceability System for China Based on RFID & Blockchain Technology," in *Proc. 13th Int. Conf. Service Syst. Service Mgmt.*, 2016, pp. 1–6.

[4] IBM, "IBM Food Trust," IBM Corporation, 2021. [Online]. Available: https://www.ibm.com/blockchain/solutions/food-trust

[5] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentrlised Application Platform," *Ethereum White Paper*, 2014.

[6] P. Helo and Y. Hao, "Blockchains in Operations and Supply Chains: A Model and Reference Implmentation," *Comput. Ind. Eng.*, vol. 136, pp. 242–251, Oct. 2019.

[7] OpenZeppelin, "OpenZeppelin Contracts," 2023. [Online]. Available: https://openzeppelin.com/contracts/

[8] Nomic Fondation, "Hardhat: Ethereum Development Environment," 2023. [Online]. Available: https://hardhat.org/